

PIPE-Cypher: Automatic Enterprise Benchmark Generation for Text-to-Cypher Systems

Anonymous ACL submission

Abstract

Enterprises increasingly use property graphs and Cypher to analyze fraud, risk, identity, customer, and operational data. Public Text2Cypher benchmarks rarely reflect private schemas, terminology, access patterns, or difficulty distributions. We present PIPE-Cypher, a local-model pipeline for generating organization-specific natural-language-to-Cypher benchmarks. PIPE-Cypher combines schema introspection, reverse-query grounding, constrained Cypher generation, deterministic read-only and schema validation, execution feedback, repair, diversity controls, and LLM-judge review. The system is designed for industry settings where benchmark generation must avoid paid APIs, preserve data governance, and remain repeatable as graphs evolve.

1 Introduction

Property graphs encode enterprise relationships directly: account transfers, identity entitlements, access paths, customer interactions, and fraud rings. Cypher is a common query language for these graphs. As LLMs become natural-language interfaces for graph analytics, organizations need reliable benchmarks for natural-language-to-Cypher systems on their own schemas.

Static public benchmarks are insufficient for this setting. They cannot contain private labels, relationship types, categorical values, or domain-specific wording, and they do not track schema evolution. PIPE-Cypher addresses this gap by generating private, repeatable, execution-grounded NL-Cypher benchmarks.

2 Related Work

Recent Text2Cypher resources, including Mind the Query (Chauhan et al., 2025), SyntheT2C (Zhong et al., 2025), Auto-Cypher (Tiwari et al., 2025), and Text2Cypher (Ozsoy et al., 2025), show that

high-quality Cypher data requires more than asking an LLM for query pairs. These works motivate schema grounding, execution checks, synthetic generation, verification, and complexity-aware evaluation. PIPE-Cypher differs by targeting private enterprise benchmark generation as the primary artifact: organizations bring their own graph, run local models, enforce Cypher constraints, and receive an executable benchmark with diversity and judge metadata.

Text-to-SQL benchmarks such as Spider 2.0 (Lei et al., 2024) and BIRD (Li et al., 2023) motivate realistic database tasks, execution-based evaluation, and enterprise workflow complexity. CIKM AutoQuery (Zheng et al., 2024) motivates strategy-level workload generation analysis and separating workload quality from model quality. LDBC FinBench (Qi et al., 2023) and SNB (Püroja et al., 2023) provide graph workloads suitable for industry-oriented evaluation.

For generated benchmark quality, PIPE-Cypher combines text-generation diversity diagnostics with text-to-query structure metrics. Distinct-n (Li et al., 2016) and self-BLEU-style redundancy checks (Zhu et al., 2018) capture lexical repetition, while schema coverage, query-signature diversity, structural feature rates, and normalized entropy over graph/category/difficulty cells capture properties that matter for executable Cypher benchmarks.

3 Method

PIPE-Cypher has six stages: schema profiling, template generation, reverse Cypher grounding, constrained Cypher generation and repair, deterministic validation and execution, and LLM-judge review. Deterministic checks enforce read-only safety, syntax shape, schema-visible labels and properties, explicit relationship types, observed relationship directions, schema-provided categorical values, and non-empty execution where required. A

lightweight Cypher analyzer extracts return aliases, variables, labels, relationship observations, risky constructs, and rewrite skip reasons so normalization is auditable rather than a silent string edit. The direction gate interprets both outgoing and incoming Cypher arrow syntax before schema checking, and rejects undirected relationship patterns so directionality is an executable constraint rather than only a prompt instruction.

4 Implementation

The implementation is a Python package with Neo4j as the experimental backend. The paper frames the contribution around Cypher and property graphs rather than a single vendor. Local models are served through a vLLM/OpenAI-compatible endpoint on ds-serv6, using Qwen3.5-9B for generation and judging.

All reported generation, judging, and downstream evaluation runs use a local Qwen3.5-9B endpoint. This keeps the study aligned with an enterprise deployment pattern in which the benchmark pipeline runs inside the organization’s compute boundary without paid generation APIs.

The built-in FinBench profile is grounded in the public datagen snapshot export and includes typed node and relationship properties. Live schema inspection also records low-cardinality string values as categorical constraints for prompting and validation. The generated Neo4j import script uses uniqueness constraints for nodes and creates, rather than merges, transaction relationships so repeated account-to-account events are preserved. The SNB reference profile is grounded in the official Neo4j/Cypher headers and read-query files.

For generation, PIPE-Cypher uses seeded workload templates with deterministic reverse-binding queries and a mixed mode that prepends proven workload seeds to LLM-proposed templates. Every run records accepted and rejected candidates for yield analysis. Retrieved few-shot examples replace graph-specific values with typed placeholders, preserving query structure while reducing tenant-value leakage and memorized entity reuse. A dependency-light value grounder adds typed prompt annotations for categorical values and reverse-bound entities, covering punctuation variants, possessives, plurals, synonyms, name partials, and small typos.

For LLM-judge review, the implementation slices schema context to the labels, relationship

types, and properties used by the candidate query. This preserves validation against the full schema while keeping local 9B prompts within context limits on larger schemas.

The deterministic Cypher layer borrows from production lessons in the BalkanID Cypher system: schema-only prompting, exact matching, relationship direction discipline, RETURN DISTINCT, reserved variable rejection, categorical values, required contextual return columns, fuzzy value annotations, placeholderized retrieval examples, and parser-aware rewrite boundaries. PIPE-Cypher records parser-style structure features and skips rewrites for risky constructs such as UNION, CALL, UNWIND, WHERE EXISTS, multiple WHERE clauses, or reserved variables.

We also estimate seed-template capacity before full generation. This check caught and fixed a scale blocker: reverse-binding execution was hard-capped to 10 rows, and no-slot negation/ranking seeds could not support full category targets. PIPE-Cypher now uses the configured binding limit and includes slotted negation and ranking seeds for both graphs.

For arbitrary enterprise onboarding, PIPE-Cypher includes a deployment template for a company’s own graph, read-only credentials, local model endpoint, schema introspection, privacy policy, dry run, scaled run, audit, and redacted export. Configurable value-sampling policies bound which low-cardinality graph values enter schema prompts, and redacted exports replace quoted literals, entity values, and string-valued result samples with stable placeholders for broader internal review.

5 Experiments

The generated benchmark contains 3,000 accepted examples: 2,000 from LDBC FinBench and 1,000 from LDBC SNB. Categories include simple retrieval, complex retrieval, simple aggregation, complex aggregation, boolean existence, negation/difference, path/temporal transaction, and ranking/top-k.

6 Results

The full live run produced 3,000 accepted examples from 4,777 candidates using local Qwen3.5-9B for generation and judging. Category-specific recovery top-ups filled the only under-target categories from the initial sequential run. Every exported example

Gate	Evidence checked	Failure mode caught
Read-only safety	blocked Cypher write/admin tokens	destructive or operational queries
Schema validity	labels, relationship types, properties, categorical values	hallucinated graph vocabulary or values
Direction validity	observed relationship directions	reversed or invalid traversals
Cypher analysis and normalization	structure extraction, rewrite safety, RETURN DISTINCT	unsafe rewrites or duplicate/noisy rows
Question constraints	quoted values use exact literals	fuzzy matching of exact user values
Execution	query runs and returns rows	syntactic validity without answerability
LLM judge	ambiguity, semantic alignment, schema use	executable but semantically weak examples

Table 1: PIPE-Cypher candidate acceptance gates.

Graph	Target/category	Binding limit	Min seed capacity	All categories meet target
FinBench	250	300	300	Yes
SNB	125	200	200	Yes

Table 2: Built-in seed-template capacity after removing the reverse-binding 10-row cap and adding slotted negation/ranking seeds. Capacity is a theoretical scale-readiness check; final examples still must pass validation, execution, diversity, and judge gates.

Setting	Value
Accepted examples	3,000
Primary graph	LDBC FinBench, 2,000 examples
Secondary graph	LDBC SNB, 1,000 examples
Categories	8 balanced categories, 375 each
Generation/judge model	Local Qwen3.5-9B
Execution backend	Neo4j Community, two live databases
Judge audit packet	80 sampled accepted/rejected pairs

Table 3: Full live experimental setup used for the generated benchmark artifact.

passed read-only, syntax, schema, execution, non-empty result, and judge gates.

The accepted full-run records were exported into a benchmark package with stable example identifiers, train/dev/test splits, result samples, gate metadata, aggregate statistics, and a manifest hash for artifact tracking. The export contains 3,000 accepted examples: 2,000 FinBench, 1,000 SNB, and 375 accepted examples in every planned category.

We report diversity as a diagnostic, not only as a design target. The full export has perfect category balance and near-perfect difficulty balance, uses 1,115 unique grounded entity values, and exactly quotes grounded values in 82.6% of examples with entity bindings. The reported local-model run still has low query-signature diversity because seeded graph-grounded templates are doing substantial work. This is useful evidence for industry deployment: the artifact is balanced and executable, while the appendix makes template and value concentration visible rather than hiding it.

The full-run failure taxonomy is concentrated

Graph	Candidates	Accepted	Acceptance	Categories at target
FinBench	3,404	2,000	0.588	8/8
SNB	1,373	1,000	0.728	8/8
Total	4,777	3,000	0.628	16/16

Table 4: Full live generation with local Qwen3.5-9B. Candidate counts include the initial sequential run and category-specific recovery top-ups.

Export artifact	Examples	FinBench	SNB	Train	Dev	Test
Live full benchmark	3,000	2,000	1,000	2,408	296	296

Table 5: Accepted live full benchmark package with stable IDs, gate metadata, result samples, statistics, and manifest hash 8bc79a53a06b291a.

in diversity/duplicate controls during recovery and empty execution results; only 2/4,777 candidates were schema-invalid after deterministic Cypher governance.

For judge calibration, the released artifacts include an 80-row audit packet sampled from accepted and rejected full-run candidates, with a 40/40 judge accept/reject split and coverage over both graphs and all eight categories. A completed human audit shows 80.0% agreement, Cohen’s $\kappa = 0.60$, judge precision and specificity of 1.00, recall of 0.714, and no false accepts in the sample. The judge is therefore conservative: it protects accepted-example quality while rejecting some human-approved candidates and reducing yield. Human labels are used only for post-hoc calibration, not as a generation gate.

Finally, we ran a downstream Text2Cypher evaluation using local Qwen3.5-9B on the exported full test split. The model saw schema text and the natural-language question, generated Cypher, and was evaluated by live execution against the corresponding FinBench or SNB database. The result verifies that the exported artifact can drive downstream model evaluation and gives a difficulty signal: the zero-shot 9B baseline solves simple

Artifact property	Value
Categories	8 balanced categories
FinBench/category	250
SNB/category	125
Difficulty split	1,521 easy / 1,479 medium
Unique labels / rel. types	7 / 9
Read/syntax/schema/exec/judge gates	3,000/3,000

Table 6: Distribution and gate summary for the exported full benchmark artifact.

Split	Examples	Parse	Schema	Exec. success	Exec. acc.	Answer F1
Live full test	296	0.959	0.905	0.622	0.189	0.189

Table 7: Downstream Text2Cypher evaluation for local Qwen3.5-9B on the exported full benchmark test split.

aggregation examples but struggles on more operational categories.

7 Industry Use

PIPE-Cypher is intended to be rerun when an enterprise graph changes. The generated artifact records the schema snapshot, graph profile, model identifier, validation gates, execution samples, judge scores, difficulty features, and source run for every example. Long-running jobs can be monitored from JSONL records and recovered with category-specific top-up runs that reject questions accepted in earlier runs. This design supports private benchmark refreshes without exposing schemas or values to paid APIs, while still leaving audit hooks for data owners to sample accepted and rejected examples.

The deployment path is designed for arbitrary enterprise schemas: teams provide read-only graph credentials, introspect labels/relationships/properties, choose a bounded categorical-value sampling policy, connect a local model endpoint, run a dry pass, scale generation, calibrate the judge, and export either raw internal artifacts or redacted review artifacts. This makes the FinBench/SNB study a reproducible evaluation setting rather than a hard-coded graph assumption.

8 Conclusion

PIPE-Cypher provides a practical path for enterprises to create private, executable, balanced NL-to-Cypher benchmarks. The pipeline combines schema introspection, constrained generation, deterministic validation, execution feedback, diversity control, and automated judge review.

Limitations

Execution validity does not guarantee semantic correctness. The completed audit suggests a conservative judge with no false accepts in the labeled sample, but larger audits may reveal additional failure modes. Generated artifacts may contain private schema details or sampled values, so organizations should apply privacy redaction and normal data governance controls before broad sharing.

Ethics Statement

PIPE-Cypher is designed for private benchmark generation under local-model deployment constraints. Organizations using it should restrict benchmark artifacts to authorized users, review sampled values for sensitive content, and document whether judge calibration relied on human annotators.

References

- Vashu Chauhan, Shobhit Raj, Shashank Mujumdar, Avirup Saha, and Anannay Jain. 2025. [Mind the query: A benchmark dataset towards Text2Cypher task](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1890–1905, Suzhou, China. Association for Computational Linguistics.
- Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. 2024. [Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows](#). *Preprint*, arXiv:2411.07763.
- Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. [Can LLM already serve as a database interface? a BIG bench for large-scale database grounded text-to-SQLs](#). In *Advances in Neural Information Processing Systems*.
- Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and Bill Dolan. 2016. [A diversity-promoting objective function for neural conversation models](#). In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 110–119, San Diego, California. Association for Computational Linguistics.
- Makbule Gulcin Ozsoy, Leila Messallem, Jon Besga, and Gianandrea Minneci. 2025. [Text2Cypher: Bridging natural language and graph databases](#). In *Proceedings of the Workshop on Generative AI and*

Knowledge Graphs, pages 100–108, Abu Dhabi, UAE. International Committee on Computational Linguistics.

David P

üroja, Jack Waudby, Peter Boncz, and G

'abor Sz

'arnyas. 2023. [The LDBC social network benchmark interactive workload v2: A transactional graph query benchmark with deep delete operations](#). *Preprint*, arXiv:2307.04820.

Shipeng Qi, Heng Lin, Zhihui Guo, G

'abor Sz

'arnyas, Bing Tong, Yan Zhou, Bin Yang, Jian-song Zhang, Zheng Wang, Youren Shen, Changyuan Wang, Parviz Peiravi, Henry Gabb, and Ben Steer. 2023. [The LDBC financial benchmark](#). *Preprint*, arXiv:2306.15975.

Aman Tiwari, Shiva Krishna Reddy Malay, Vikas Yadav, Masoud Hashemi, and Sathwik Tejaswi Madhusudhan. 2025. [Auto-cypher: Improving LLMs on cypher generation via LLM-supervised generation-verification framework](#). In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2: Short Papers*, pages 623–640, Albuquerque, New Mexico. Association for Computational Linguistics.

Xiuwen Zheng, Arun Kumar, and Amarnath Gupta. 2024. [Generating cross-model analytics workloads using LLMs](#). In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 4303–4307. ACM.

Zijie Zhong, Linqing Zhong, Zhaoze Sun, Qingyun Jin, Zengchang Qin, and Xiaofan Zhang. 2025. [Syn-theT2C: Generating synthetic data for fine-tuning large language models on the Text2Cypher task](#). In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 672–692, Abu Dhabi, UAE. Association for Computational Linguistics.

Yaoming Zhu, Sidi Lu, Lei Zheng, Jiaxian Guo, Weinan Zhang, Jun Wang, and Yong Yu. 2018. [Texygen: A benchmarking platform for text generation models](#). In *The 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1097–1100. ACM.

A Additional Results

B Claim–Evidence Map

This section maps the main paper claims to the strongest current evidence and to the remaining risks. This is intended as a reviewer-facing audit surface: claims with pending evidence remain marked as such rather than being folded into the main results.

Metric	Value
Question Distinct-1	0.041
Question Distinct-2	0.088
Question self-BLEU-2 (sampled)	0.826
Unique query-signature ratio	0.010
Top query-signature share	0.083
Category normalized entropy	1.000
Graph-category normalized entropy	0.980
Difficulty normalized entropy	1.000
Label coverage	0.412
Relationship-type coverage	0.375
Property-name coverage	0.407
Unique grounded-value ratio	0.373
Grounded values exactly quoted	0.826
Aggregation / negation / ordering rates	0.500 / 0.125 / 0.125

Table 8: Diversity diagnostics for the full exported benchmark. Distinct-n follows text-generation usage; self-BLEU is lower when questions are less redundant; grounded-value metrics are aggregate-only and do not list raw values.

Failure bucket	Count	Share of rejected
Diversity/duplicate control	1,335	0.751
Empty result	374	0.210
Judge semantic reject	66	0.037
Schema invalid	2	0.001

Table 9: Failure taxonomy over full-run generation candidates before benchmark export. Accepted examples are excluded from the bucket shares.

- Claim 1.** PIPE-Cypher can generate a private, executable NL-to-Cypher benchmark over enterprise-style property graphs using local models.

Evidence. The full local Qwen3.5-9B run exported 3,000 accepted examples: 2,000 FinBench and 1,000 SNB, with 375 examples in every planned workload category and all accepted examples passing read-only, syntax, schema, execution, non-empty-result, and judge gates.

Key artifacts. benchmark export; manifest snapshot; paper table: benchmark export; paper table: full results

Status. Supported by live local-model evidence.

Risk. Generation quality may change with private enterprise schemas beyond FinBench/SNB.

- Claim 2.** Cypher-specific governance makes generated benchmark candidates safer and more auditable than prompt-only generation.

Evidence. The pipeline validates read-only

Setting	Graph	Records	Accepted	Acceptance	Categories at target
Unconstrained LLM	FinBench	0	0	0.000	0/8
Unconstrained LLM	SNB	0	0	0.000	0/8
Reverse-only	FinBench	400	400	1.000	8/8
Reverse-only	SNB	402	400	0.995	8/8
Validators+repair	FinBench	400	400	1.000	8/8
Validators+repair	SNB	402	400	0.995	8/8
No retrieval	FinBench	429	400	0.932	8/8
No retrieval	SNB	402	400	0.995	8/8
No rewrite	FinBench	423	400	0.946	8/8
No rewrite	SNB	402	400	0.995	8/8
No LLM judge	FinBench	401	400	0.998	8/8
No LLM judge	SNB	406	400	0.985	8/8
Full PIPE-Cypher	FinBench	416	400	0.962	8/8
Full PIPE-Cypher	SNB	402	400	0.995	8/8

Table 10: Live target-50 ablation evidence with local Qwen3.5-9B. Each graph run targets 50 accepted examples per category.

Setting	Graph	Read-only	Syntax	Schema	Exec.	Judge
Unconstrained LLM	FinBench	0.000	0.000	0.000	0.000	0.000
Unconstrained LLM	SNB	0.000	0.000	0.000	0.000	0.000
Reverse-only	FinBench	1.000	1.000	1.000	1.000	1.000
Reverse-only	SNB	1.000	1.000	0.995	0.995	0.995
Validators+repair	FinBench	1.000	1.000	1.000	1.000	1.000
Validators+repair	SNB	1.000	1.000	0.995	0.995	0.995
No retrieval	FinBench	1.000	1.000	1.000	1.000	0.932
No retrieval	SNB	1.000	1.000	0.995	0.995	0.995
No rewrite	FinBench	1.000	1.000	1.000	1.000	0.946
No rewrite	SNB	1.000	1.000	0.995	0.995	0.995
No LLM judge	FinBench	1.000	1.000	1.000	1.000	0.998
No LLM judge	SNB	1.000	1.000	0.995	0.995	0.985
Full PIPE-Cypher	FinBench	1.000	1.000	1.000	1.000	0.962
Full PIPE-Cypher	SNB	1.000	1.000	0.995	0.995	0.995

Table 11: Quality-gate rates for the live target-50 ablation suite. Rates are computed over all generated records in each graph/setting.

safety, schema-visible labels, node and relationship properties, relationships, observed relationship direction, categorical values, contextual return columns, parser-style structure, execution success, and LLM-judge review. In the full run, only 2 of 4,777 candidates were schema-invalid after deterministic governance, and all 3,000 exported examples passed the deterministic and judge gates.

Key artifacts. code: validator.py; code: cypher_parser.py; code: question_constraints.py; snapshot: failure taxonomy; +1 more

Status. Supported by code, tests, and full-run failure taxonomy.

Risk. Semantic correctness still depends on execution evidence and judge review; the completed 80-row audit is a calibration sample rather than an exhaustive proof.

- Claim 3.** Reverse grounding and structured workload seeds are designed to improve reliability under local-model constraints.

Evidence. The target-50 FinBench/SNB ablation suite completed 14/14 graph/variant cells, reached every category target for all non-empty/non-unconstrained runs, and passed

Audit packet property	Value
Rows	80
Judge accept / reject	40 / 40
FinBench / SNB rows	48 / 32
Easy / medium rows	26 / 54
Labeled rows	80
Agreement / Cohen’s κ	0.800 / 0.600
Judge precision / recall	1.000 / 0.714
Judge specificity	1.000
False accepts / false rejects	0 / 16

Table 12: Post-hoc judge calibration from the completed 80-row human audit. Labels are used only to calibrate the automated judge, not as a generation gate.

Metric	Point	95% CI	SE	N
Parse valid	0.959	[0.936, 0.980]	0.012	296
Schema valid	0.905	[0.872, 0.939]	0.017	296
Execution success	0.622	[0.564, 0.676]	0.028	296
Execution accuracy	0.189	[0.145, 0.233]	0.022	296
Answer F1	0.189	[0.149, 0.236]	0.023	296

Table 13: Bootstrap uncertainty for downstream Text2Cypher evaluation on the full exported test split. Intervals use row-level resampling with a fixed seed and are intended for appendix reporting.

the paper-readiness audit with logs, model IDs, code revision, run summaries, and gate-rate availability. Appendix-only tables plus yield and gate-rate figures report the audited target-50 evidence while the target-100 follow-up and seeded target-50 repeat continue for stronger final reliability and variance claims.

Key artifacts. script: run live ablation suite; script: monitor remote ablation suite; script: monitor remote ablation queue; script: collect remote ablation suite; +9 more

Status. Supported by audited target-50 appendix evidence; target-100 and seeded target-50 repeat runs are still pending for stronger final reliability claims.

Risk. A single target-50 suite is the minimum publishable ablation scale, not the strongest possible evidence. If target-100 or repeated-seed runs complete, final claims should use those sensitivity results instead of relying only on target-50.

- Claim 4.** The benchmark controls category and difficulty balance while exposing residual diversity limits.

Downstream outcome	Count	Share of incorrect
Answer mismatch	128	0.533
Execution failed	72	0.300
Schema invalid	28	0.117
Parse invalid	12	0.050

Table 14: Downstream Text2Cypher failure taxonomy for local Qwen3.5-9B on the full exported test split. Shares exclude exact-answer matches.

Evidence. The full export has perfect category balance, planned 2:1 FinBench/SNB graph mix, and a near-even easy/medium split. Diversity diagnostics report Distinct-n, sampled self-BLEU-2, query-signature diversity, normalized entropy, schema coverage, structural feature rates, and aggregate value-grounding diagnostics. The full export uses 1,115 unique grounded entity values and exactly quotes grounded values in 82.6% of examples with entity bindings, making template and value concentration visible instead of hidden.

Key artifacts. snapshot: diversity report; paper table: diversity; paper figure: diversity diagnostics; paper figure: full export distribution

Status. Supported by full-export statistics and diversity diagnostics.

Risk. Low query-signature diversity in the reported local-model run remains a limitation and ablation target.

- Claim 5.** The generated benchmark is useful for downstream Text2Cypher evaluation.

Evidence. A zero-shot local Qwen3.5-9B downstream evaluation over the 296-example full test split reports parse validity, schema validity, execution success, execution accuracy, answer F1, per-category breakdowns, fixed-seed bootstrap uncertainty intervals, and a row-level error taxonomy. The model reaches 0.189 execution accuracy with a 95% bootstrap interval of [0.145, 0.233] and 0.622 execution success with interval [0.564, 0.676]. The error taxonomy classifies 240 incorrect rows into answer mismatches, execution failures, schema invalid predictions, and parse-invalid predictions, showing the benchmark can discriminate easy aggregation from harder operational categories and can explain model failure modes.

Key artifacts. downstream summary; snap-

shot: downstream uncertainty; snapshot: downstream error report; research note: downstream evaluation; +6 more

Status. Supported by live downstream evaluation against FinBench/SNB databases, with bootstrap uncertainty computed from row-level outputs.

Risk. Only one downstream model has been evaluated so far.

- Claim 6.** Automated LLM-judge review can replace human-in-the-loop generation gates while still exposing judge risk.

Evidence. The pipeline uses deterministic validation plus LLM-judge review as the generation gate. A post-hoc 80-row judge audit packet preserves 40 judge accepts, 40 judge rejects, both graphs, and all eight categories. The completed human audit reports 80.0% agreement, Cohen’s kappa 0.60, balanced accuracy 0.857, judge precision 1.00, judge specificity 1.00, judge recall 0.714, zero false accepts, and 16 false rejects. The analyzer requires complete labels for paper-facing calibration, and the tracked summary avoids committing raw value-bearing audit rows.

Key artifacts. code: judge.py; code: calibration.py; code: judge_audit_packet.py; script: create judge annotation sheets; +6 more

Status. Supported by completed human audit summary.

Risk. The judge appears conservative on this sample; larger audits or different enterprise schemas may reveal false accepts.

- Claim 7.** PIPE-Cypher supports arbitrary enterprise schema onboarding with configurable privacy redaction and value-sampling policies.

Evidence. The repo includes an enterprise deployment guide, an enterprise template config, privacy config fields, schema introspection that reads the categorical-value threshold from config, deterministic redaction helpers, and a redacted benchmark export CLI. The redaction policy replaces quoted Cypher literals, question mentions, entity values, and string-valued result samples with stable placeholders while preserving query structure and gate metadata.

525	<i>Key artifacts.</i> research note: enterprise deployment guide; enterprise_template.yaml; code: privacy.py; script: redact benchmark export; +2 more	569
526		570
527		571
528		572
529	<i>Status.</i> Supported by docs, config, code, and deterministic tests.	573
530		574
531	<i>Risk.</i> The strongest industry validation would still come from running the pipeline on a real or anonymized enterprise graph beyond LDBC.	575
532		576
533		
534		
535	C Prompt Contracts	
536	PIPE-Cypher treats prompts as versioned implementation artifacts. The list summarizes the prompt contracts used for generation, repair, judging, and downstream evaluation; hashes fingerprint the full prompt constants in the codebase.	
537		
538		
539		
540		
541	1. Template generation. Stage: Workload proposal. SHA-256: 10b25b.	
542		
543	<i>Contract.</i> Schema-only labels, relationships, properties, and categorical values; Realistic enterprise analyst wording; At most two typed slots and JSON-only output	
544		
545		
546		
547	2. Reverse binding. Stage: Graph grounding. SHA-256: 48e949.	
548		
549	<i>Contract.</i> Read-only	
550	MATCH/WHERE/RETURN DISTINCT/LIMIT only; Slot variables named exactly as requested; Forward relationship directions from the schema	
551		
552		
553		
554	3. Cypher generation. Stage: Candidate query. SHA-256: 61c557.	
555		
556	<i>Contract.</i> Only schema-visible constructs and observed directions; RETURN DISTINCT for set returns and exact equality for quoted values; Context columns, categorical hints, placeholderized retrieval, and no writes	
557		
558		
559		
560		
561	4. Repair. Stage: Validation feedback. SHA-256: 56c419.	
562		
563	<i>Contract.</i> Preserve question intent while fixing validation or execution issues; Keep query read-only and schema-grounded; Return only corrected Cypher	
564		
565		
566		
567	5. LLM judge. Stage: Quality gate. SHA-256: 421c7b.	
568		
	<i>Contract.</i> Inputs include question, Cypher, schema slice, execution rows, and validation summary; Strict JSON scores for ambiguity, semantic alignment, schema use, and difficulty; Categorical values constrain query literals, not observed result-row values; Pass only useful, unambiguous enterprise benchmark examples	
	6. Downstream Text2Cypher. Stage: Model evaluation. SHA-256: 4c07ff.	
	<i>Contract.</i> Read-only Cypher only; Schema-visible constructs and exact direction preservation; RETURN DISTINCT, count/ranking rules, and no explanations	
	D Representative Accepted Examples	
	The examples below are selected in stable identifier order from the tracked full-export snapshot, one per graph/category cell when available. They show the natural-language question, accepted Cypher, structural tags, gate status, and a bounded execution-result sample.	
	1. FinBench / Boolean Existence / easy. <i>Question:</i> Does account '187743809466009406' have any outgoing transfer?	
	<pre>MATCH (src:Account {accountId: '187743809466009406'}) -[:TRANSFER_TO]-> (:Account) RETURN DISTINCT COUNT(DISTINCT src) > 0 AS HasOutgoingTransfer</pre>	
	<i>Structure:</i> single_hop, aggregation.	
	<i>Relationships:</i> TRANSFER_TO.	
	<i>Gates:</i> RO/Syn/Schema/Exec/Judge.	
	<i>Result sample:</i> {HasOutgoingTransfer: True}; observed rows: 1	
	2. FinBench / Complex Aggregation / medium. <i>Question:</i> What is the total transferred amount from accounts owned by person 'Gwar'?	
	<pre>MATCH (p:Person {personName: 'Gwar'}) -[:OWN_ACCOUNT]-> (src:Account)-[:TRANSFER_TO]-> (:Account) RETURN DISTINCT SUM(t.amount) AS TotalTransferredAmount</pre>	
	<i>Structure:</i> join_heavy, aggregation.	
	<i>Relationships:</i> OWN_ACCOUNT, TRANSFER_TO.	
	<i>Gates:</i> RO/Syn/Schema/Exec/Judge.	

619	<i>Result sample:</i> {TotalTransferredAmount:	<i>Structure:</i> join_heavy, path, bounded_result.	676
620	14972318.41}; observed rows: 1		
621	3. FinBench / Complex Retrieval / easy. Question:	<i>Relationships:</i> OWN_ACCOUNT, TRANS-	677
622	Which accounts received transfers from	FER_TO.	678
623	accounts owned by person 'Zof'?	<i>Gates:</i> RO/Syn/Schema/Exec/Judge.	679
624	<pre>MATCH (p:Person {personName:</pre>	<i>Result sample:</i> {AccountId:	680
625	'Zof'})	4687402787162554854, AccountType:	681
626	-[:OWN_ACCOUNT]-> (src:Account)	credit card, IsBlocked: False}; observed rows:	682
627	-[:TRANSFER_TO]-> (dst:Account)	4	683
628	RETURN DISTINCT dst.accountId AS		
629	AccountId, dst.accountType AS		
630	AccountType, dst.isBlocked AS		
631	IsBlocked		
632	LIMIT 300		
633	<i>Structure:</i> join_heavy, bounded_result.		
634	<i>Relationships:</i> OWN_ACCOUNT, TRANS-	6. FinBench / Ranking Topk / medium. Question:	684
635	FER_TO.	For accounts owned by person 'Barry',	685
636	<i>Gates:</i> RO/Syn/Schema/Exec/Judge.	which account sent the highest total transfer	686
637	<i>Result sample:</i> {AccountId:	amount?	687
638	4687402787162554854, AccountType:		
639	credit card, IsBlocked: False}; observed rows:	<pre>MATCH (p:Person {personName:</pre>	688
640	3	'Barry'})	689
641		-[:OWN_ACCOUNT]->	690
642	4. FinBench / Negation Difference / medium.	(src:Account)-[t:TRANSFER_TO]->	691
643	<i>Question:</i> Which accounts owned by person	(:Account)	692
644	'Kant' have not sent any transfers?	WITH src, SUM(t.amount) AS	693
645	<pre>MATCH (p:Person {personName:</pre>	totalAmount	694
646	'Kant'})	RETURN DISTINCT src.accountId AS	695
647	-[:OWN_ACCOUNT]-> (a:Account)	AccountId, src.accountType AS	696
648	WHERE NOT (a) -[:TRANSFER_TO]->	AccountType, src.isBlocked AS	697
649	(:Account)	IsBlocked,	698
650	RETURN DISTINCT a.accountId AS	totalAmount	699
651	AccountId, a.accountType AS	ORDER BY totalAmount DESC	700
652	AccountType,	LIMIT 1	701
653	a.isBlocked AS IsBlocked		
654	LIMIT 300	<i>Structure:</i> join_heavy, aggregation, or-	702
655	<i>Structure:</i> join_heavy, negation,	der_rank, bounded_result.	703
656	bounded_result.	<i>Relationships:</i> OWN_ACCOUNT, TRANS-	704
657	<i>Relationships:</i> OWN_ACCOUNT, TRANS-	FER_TO.	705
658	<i>Gates:</i> RO/Syn/Schema/Exec/Judge.	<i>Gates:</i> RO/Syn/Schema/Exec/Judge.	706
659	<i>Result sample:</i> {AccountId:	<i>Result sample:</i> {AccountId:	707
660	4739757132830738341, AccountType:	4732438783436260772, AccountType:	708
661	merchant account, IsBlocked: False};	internet account, IsBlocked: False}; observed	709
662	observed rows: 1	rows: 1	710
663	5. FinBench / Path Temporal / medium. Question:	7. FinBench / Simple Aggregation / easy. Question:	711
664	Which accounts can receive money	How many accounts are owned by per-	712
665	within two transfer hops from accounts owned	son 'Kaewsuktae'?	713
666	by person 'Sossamon'?	<pre>MATCH (p:Person {personName:</pre>	714
667	<pre>MATCH (p:Person {personName:</pre>	'Kaewsuktae'}) -[:OWN_ACCOUNT]->	715
668	'Sossamon'}) -[:OWN_ACCOUNT]->	(a:Account)	716
669	(src:Account) -[:TRANSFER_TO*1..2]->	RETURN DISTINCT COUNT(DISTINCT a) AS	717
670	(dst:Account)	AccountCount	718
671	RETURN DISTINCT dst.accountId AS	<i>Structure:</i> single_hop, aggregation.	719
672	AccountId, dst.accountType AS	<i>Relationships:</i> OWN_ACCOUNT.	720
673	AccountType, dst.isBlocked AS	<i>Gates:</i> RO/Syn/Schema/Exec/Judge.	721
674	IsBlocked	<i>Result sample:</i> {AccountCount: 1}; observed	722
675	LIMIT 300	rows: 1	723
		8. FinBench / Simple Retrieval / easy. Question:	724
		Which accounts are owned by person	725
		'Barry'?	726

727	MATCH (p:Person {personName:	783	Structure: join_heavy, bounded_result.
728	'Barry'}})		
729	-[:OWN_ACCOUNT]-> (a:Account)	784	Relationships: CONTAINER_OF,
730	RETURN DISTINCT a.accountId AS	785	HAS_MEMBER, HAS_TAG.
731	AccountId, a.accountType AS		
732	AccountType,	786	Gates: RO/Syn/Schema/Exec/Judge.
733	a.isBlocked AS IsBlocked		
734	LIMIT 300	787	Result sample: {PersonId: 4398046511220};
735	Structure: single_hop, bounded_result.	788	observed rows: 19
736	Relationships: OWN_ACCOUNT.		
737	Gates: RO/Syn/Schema/Exec/Judge.	789	12. SNB / Negation Difference / medium. Ques-
738	Result sample: {AccountId:	790	tion: Which members of forum 'Wall of Celso
739	4732438783436260772, AccountType:	791	Oliveira' have not liked any post?
740	internet account, IsBlocked: False}; observed		
741	rows: 1	792	MATCH (forum:Forum {title: 'Wall of
742	9. SNB / Boolean Existence / medium. Ques-	793	Celso Oliveira'}) -[:HAS_MEMBER]->
743	tion: Does person with id 6597069766828	794	(p:Person)
744	like any post?	795	WHERE NOT (p) -[:LIKES]-> (:Post)
745	MATCH (p:Person {id:	796	RETURN DISTINCT p.id AS PersonId,
746	6597069766828})	797	p.firstName AS FirstName, p.lastName
747	OPTIONAL MATCH (p) -[:LIKES]->	798	AS
748	(post:Post)	799	LastName
749	RETURN DISTINCT COUNT(post) > 0 AS	800	LIMIT 200
750	LikesAnyPost	801	Structure: join_heavy, negation,
751	Structure: single_hop, aggregation, optional.	802	bounded_result.
752	Relationships: LIKES.	803	Relationships: HAS_MEMBER, LIKES.
753	Gates: RO/Syn/Schema/Exec/Judge.	804	Gates: RO/Syn/Schema/Exec/Judge.
754	Result sample: {LikesAnyPost: True}; ob-	805	Result sample: {FirstName: K., LastName:
755	served rows: 1	806	Sen, PersonId: 94}; observed rows: 1
756	10. SNB / Complex Aggregation / medium.	807	13. SNB / Path Temporal / easy. Question:
757	Question: How many distinct posts	808	Which people are within two knows hops of
758	are in forums joined by person with id	809	person with id 4398046511136?
759	6597069766845?	810	MATCH (src:Person {id:
760	MATCH (forum:Forum) -[:HAS_MEMBER]->	811	4398046511136})
761	(p:Person {id: 6597069766845})	812	-[:KNOWS*1..2]-> (dst:Person)
762	MATCH (forum) -[:CONTAINER_OF]->	813	RETURN DISTINCT dst.id AS PersonId,
763	(post:Post)	814	dst.firstName AS FirstName,
764	RETURN DISTINCT COUNT(DISTINCT post)	815	dst.lastName
765	AS	816	AS LastName
766	JoinedForumPostCount	817	LIMIT 200
767	Structure: join_heavy, aggregation.	818	Structure: single_hop, path, bounded_result.
768	Relationships: CONTAINER_OF,	819	Relationships: KNOWS.
769	HAS_MEMBER.	820	Gates: RO/Syn/Schema/Exec/Judge.
770	Gates: RO/Syn/Schema/Exec/Judge.	821	Result sample: {FirstName: Rafael,
771	Result sample: {JoinedForumPostCount: 1};	822	LastName: Fernández, PersonId:
772	observed rows: 1	823	4398046511333}; observed rows: 25
773	11. SNB / Complex Retrieval / easy. Question:	824	14. SNB / Ranking Topk / medium. Ques-
774	Which people are members of forums contain-	825	tion: Among forums tagged 'James_K._Polk',
775	ing posts tagged 'Manuel_Noriega'?	826	which forum has the most members?
776	MATCH (forum:Forum) -[:HAS_MEMBER]->	827	MATCH (forum:Forum) -[:HAS_TAG]->
777	(p:Person), (forum)	828	(tag:Tag {name: 'James_K._Polk'})
778	-[:CONTAINER_OF]->	829	MATCH (forum) -[:HAS_MEMBER]->
779	(post:Post) -[:HAS_TAG]-> (tag:Tag	830	(person:Person)
780	{name: 'Manuel_Noriega'})	831	WITH forum, COUNT(DISTINCT person)
781	RETURN DISTINCT p.id AS PersonId	832	AS
782	LIMIT 200	833	memberCount
		834	RETURN DISTINCT forum.title AS
		835	ForumTitle, memberCount
		836	ORDER BY memberCount DESC
		837	LIMIT 1

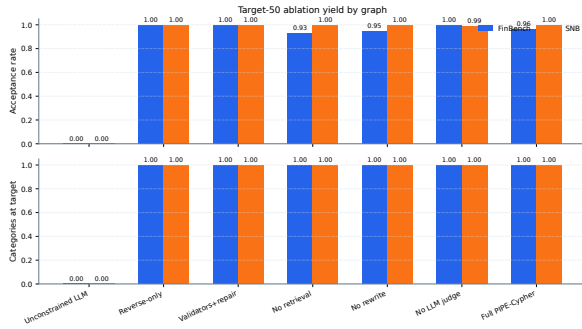


Figure 1: Audited target-50 ablation suite over FinBench and SNB. The suite reaches all graph/variant/category targets and is included as appendix evidence while the larger target-100 and seeded repeat suites continue running for stronger final reliability evidence.

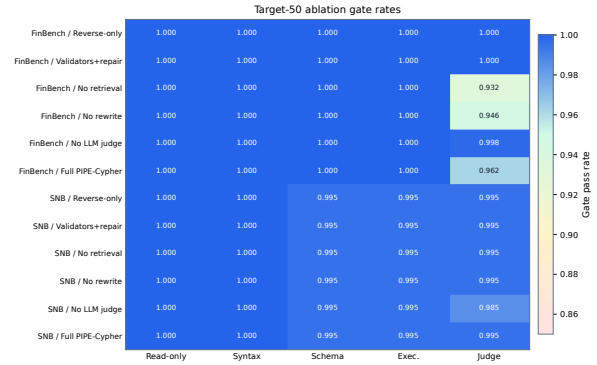


Figure 2: Gate-rate heatmap for the audited target-50 ablation suite. Deterministic read-only and syntax gates are saturated, while schema, execution, and judge rates expose the remaining quality differences across graph and pipeline variants.

Structure: join_heavy, aggregation, order_rank, bounded_result.

Relationships: HAS_MEMBER, HAS_TAG.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {ForumTitle: Wall of Javed Khan, memberCount: 33}; observed rows: 1

15. **SNB / Simple Aggregation / easy. Question:** How many posts are tagged 'Vietnam'?

```
MATCH (post:Post) -[:HAS_TAG]->
(tag:Tag
{name: 'Vietnam'})
RETURN DISTINCT COUNT(DISTINCT post)
AS
PostCount
```

Structure: single_hop, aggregation.

Relationships: HAS_TAG.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {PostCount: 1}; observed rows: 1

16. **SNB / Simple Retrieval / easy. Question:** Which post IDs did person with id 4398046511124 like?

```
MATCH (p:Person {id:
4398046511124})
-[:LIKES]-> (post:Post)
RETURN DISTINCT post.id AS PostId
LIMIT 200
```

Structure: single_hop, bounded_result.

Relationships: LIKES.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {PostId: 343597385744}; observed rows: 5

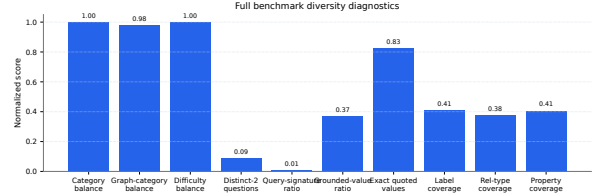


Figure 3: Diversity diagnostics for the full 3,000-example export. Category, graph-category, and difficulty balance are strong by construction; schema coverage and query-signature diversity expose remaining concentration from the seeded-template run.

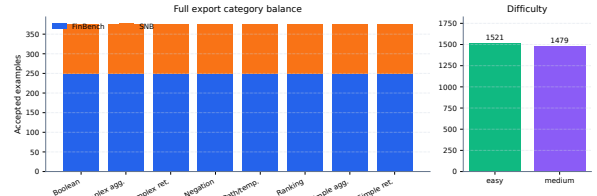


Figure 4: Full 3,000-example export distribution. The benchmark preserves the planned 2:1 FinBench/SNB graph mix while balancing all eight Cypher workload categories and maintaining a near-even easy/medium difficulty split.

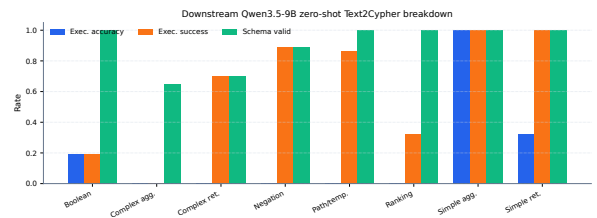


Figure 5: Downstream zero-shot Qwen3.5-9B Text2Cypher evaluation by workload category on the full test split. The breakdown separates final execution accuracy from parse/schema/execution success signals, making difficult operational categories visible.

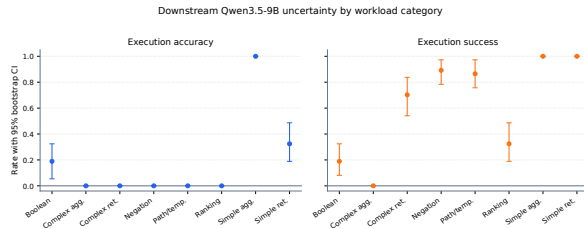


Figure 6: Bootstrap uncertainty for downstream Qwen3.5-9B Text2Cypher evaluation by workload category. Error bars show 95% row-level bootstrap intervals over the full exported test split.

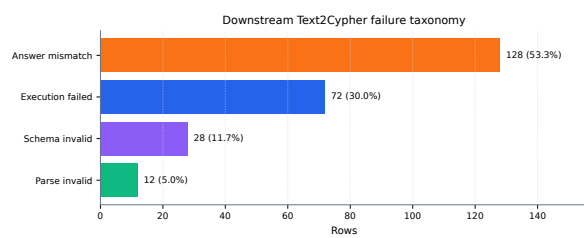


Figure 7: Downstream Text2Cypher failure taxonomy for the local Qwen3.5-9B baseline on the full test split. Exact matches are excluded so the chart exposes whether misses come from invalid Cypher, failed execution, or semantically wrong executable queries.